WEB TECHNOLOGIES PRACTICAL RECORD

Combined Questions 8–31 | JavaScript, XML, JDBC and Java Servlets

Questions, answer code, execution instructions and outputs

WEB TECHNOLOGIES PRACTICAL

Questions 8–16 | Question, Answer Code, and Execution

Note: Save each program using the filename shown. For HTML/JavaScript programs, open the HTML file in a browser. For XML questions, save the XML and DTD/XSD files in the same folder. For the Java XML parser, compile and run using a JDK.

8. JavaScript Marks, Total, Average, and Grade

Question:

Write a JavaScript program to accept marks in three subjects, calculate the total and average, and display the result and grade.

Answer Code (save as marks.html):

```
<!DOCTYPE html>
<html>
<head><title>Student Marks</title></head>
<body>
<h2>Student Marks and Grade</h2>

<script>

let m1 = Number(prompt("Enter marks in Subject 1:"));
let m2 = Number(prompt("Enter marks in Subject 2:"));
let m3 = Number(prompt("Enter marks in Subject 3:"));


let total = m1 + m2 + m3;
let average = total / 3;
let grade;


if (average >= 90) grade = "A";
else if (average >= 75) grade = "B";
else if (average >= 60) grade = "C";
else if (average >= 40) grade = "D";
else grade = "F";


document.write("Total Marks: " + total + "<br>");
document.write("Average: " + average.toFixed(2) + "<br>");
document.write("Grade: " + grade);
</script>
</body>
</html>
```

How to execute:

1. Save as marks.html.

2. Double-click the file to open it in a web browser.

3. Enter the three marks in the prompt boxes.

Sample result (marks 80, 85, 90):

Total Marks: 255

Average: 85.00

Grade: B

Expected Output / Result

Expected Output (example):
Enter marks in Subject 1: 80
Enter marks in Subject 2: 85
Enter marks in Subject 3: 90

Total Marks: 255
Average: 85.00
Grade: B

9. JavaScript Calculator Using Functions

Question:

Develop a JavaScript calculator to perform addition, subtraction, multiplication, and division using functions.

Answer Code (save as calculator.html):

```
<!DOCTYPE html>
<html>
<head><title>Calculator</title></head>
<body>
<h2>Simple Calculator</h2>
<input type="number" id="a" placeholder="First number">
<input type="number" id="b" placeholder="Second number"><br><br>
<button onclick="add()">Add</button>
<button onclick="subtract()">Subtract</button>
<button onclick="multiply()">Multiply</button>
<button onclick="divide()">Divide</button>
<p id="result"></p>

<script>
function getNumbers() {
    return [
    Number(document.getElementById("a").value),
    Number(document.getElementById("b").value)
    ];
}
function add() {
    let n = getNumbers();
    document.getElementById("result").innerHTML = "Result: " + (n[0] + n[1]);
}
function subtract() {
    let n = getNumbers();
    document.getElementById("result").innerHTML = "Result: " + (n[0] - n[1]);
}
function multiply() {
    let n = getNumbers();
    document.getElementById("result").innerHTML = "Result: " + (n[0] * n[1]);
}
function divide() {
    let n = getNumbers();
    if (n[1] === 0) {
        document.getElementById("result").innerHTML = "Cannot divide by zero";
    } else {
        document.getElementById("result").innerHTML = "Result: " + (n[0] / n[1]);
    }
}
```

```
</script>
</body>
</html>
```

How to execute:

1. Save as calculator.html.

2. Open it in a browser.

3. Enter two numbers and click an operation button.

Expected Output / Result

Expected Output (example):
First number: 12
Second number: 4

Add -> Result: 16
Subtract -> Result: 8
Multiply -> Result: 48
Divide -> Result: 3

If the second number is 0 and Divide is clicked:
Cannot divide by zero

10. Arrays and Array Methods: Largest and Smallest

Question:

Write a JavaScript program to demonstrate arrays and array methods by finding the largest and smallest elements in an array.

Answer Code (save as array.html):

```
<!DOCTYPE html>
<html>
<head><title>Array Methods</title></head>
<body>
<h2>Largest and Smallest Array Elements</h2>

<script>
let numbers = [25, 10, 45, 5, 30];


document.write("Original array: " + numbers + "<br>");


let largest = Math.max(...numbers);
let smallest = Math.min(...numbers);


document.write("Largest element: " + largest + "<br>");
document.write("Smallest element: " + smallest + "<br>");


numbers.push(50);
document.write("After push(50): " + numbers + "<br>");


numbers.sort((a, b) => a - b);
document.write("After sort(): " + numbers);
</script>
</body>
</html>
```

How to execute:

1. Save as array.html.

2. Open it in a browser.

Array methods demonstrated:

push() adds an element to the end; sort((a,b) => a-b) sorts numbers in ascending order.

Expected Output / Result

Expected Output:
Original array: 25,10,45,5,30
Largest element: 45
Smallest element: 5
After push(50): 25,10,45,5,30,50
After sort(): 5,10,25,30,45,50

11. Student Object, Destructuring, and Spread Operator

Question:

Create a student object using JavaScript and display its properties. Demonstrate object destructuring and the spread operator.

Answer Code (save as student_object.html):

```
<!DOCTYPE html>
<html>
<head><title>Student Object</title></head>
<body>
<h2>Student Object</h2>

<script>

let student = {
  name: "Ravi",

  age: 20,

  branch: "CSE",

  marks: 85

};


document.write("Name: " + student.name + "<br>");
document.write("Age: " + student.age + "<br>");
document.write("Branch: " + student.branch + "<br>");
document.write("Marks: " + student.marks + "<br><br>");


// Object destructuring
let { name, age, branch, marks } = student;
document.write("Using destructuring: " + name + ", " + age + ", " + branch + ", " + marks + "<br>");


// Spread operator
let newStudent = { ...student, marks: 90 };
document.write("Copied object with updated marks: " + JSON.stringify(newStudent));
</script>
</body>
</html>
```

How to execute:

1. Save as student_object.html.

2. Open it in a browser.

Destructuring extracts properties into variables. The spread operator copies the object properties; marks: 90 overrides the copied marks value.

Expected Output / Result

Expected Output:
Name: Ravi
Age: 20
Branch: CSE
Marks: 85

Using destructuring: Ravi, 20, CSE, 85
Copied object with updated marks: {"name":"Ravi","age":20,"branch":"CSE","marks":90}

12. Student Registration Form Validation Using Regular Expressions

Question:

Develop a JavaScript program to validate a student registration form using regular expressions for name, email, mobile number, and password.

Answer Code (save as validation.html):

```
<!DOCTYPE html>
<html>
<head><title>Registration Validation</title></head>
<body>
<h2>Student Registration</h2>
<form onsubmit="return validateForm()">
  Name: <input type="text" id="name"><br><br>
  Email: <input type="text" id="email"><br><br>
  Mobile: <input type="text" id="mobile"><br><br>
  Password: <input type="password" id="password"><br><br>
    <input type="submit" value="Register">
</form>

<script>
function validateForm() {
    let name = document.getElementById("name").value;
    let email = document.getElementById("email").value;
    let mobile = document.getElementById("mobile").value;
    let password = document.getElementById("password").value;

    if (!/^[A-Za-z ]+$/.test(name)) {
        alert("Enter a valid name");
        return false;
    }
    if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) {
        alert("Enter a valid email");
        return false;
    }
    if (!/^[0-9]{10}$/.test(mobile)) {
        alert("Mobile number must contain exactly 10 digits");
        return false;
    }
    if (password.length < 6) {
        alert("Password must contain at least 6 characters");
        return false;
    }
```

```
    alert("Registration successful");
    return false;
}
</script>
</body>
</html>
```

How to execute:

1. Save as validation.html.

2. Open it in a browser.

3. Enter details and click Register. The form checks the fields and displays an alert.

Validation rules:

Name: letters and spaces only. Email: basic email pattern. Mobile: exactly 10 digits. Password: at least 6 characters.

Expected Output / Result

Expected Output (example):
For valid details, an alert displays:
Registration successful

For invalid details, an alert displays the first validation error, for example:
Enter a valid email

13. Event Handling: Mouse, Keyboard, and Form Events

Question:

Write a JavaScript program to demonstrate event handling using mouse events, keyboard events, and form events.

Answer Code (save as events.html):

```
<!DOCTYPE html>
<html>
<head><title>Event Handling</title></head>
<body>
<h2>JavaScript Event Handling</h2>

<h3>Mouse Event</h3>
<button onclick="mouseEvent()">Click Me</button>
<p id="mouse"></p>

<h3>Keyboard Event</h3>
<input type="text" onkeydown="keyEvent()" placeholder="Press any key">
<p id="keyboard"></p>

<h3>Form Event</h3>
<form onsubmit="formEvent(event)">
    <input type="text" id="studentName" placeholder="Enter your name" required>
    <input type="submit" value="Submit">
</form>
<p id="formResult"></p>

<script>
function mouseEvent() {
    document.getElementById("mouse").innerHTML = "Mouse button clicked!";
}
function keyEvent() {
    document.getElementById("keyboard").innerHTML = "Keyboard key pressed!";
}
function formEvent(event) {
    event.preventDefault();
    let name = document.getElementById("studentName").value;
    document.getElementById("formResult").innerHTML = "Welcome " + name;
}
</script>
</body>
</html>
```

How to execute:

1. Save as events.html.
2. Open it in a browser.
3. Click the button, press a key in the text box, and submit the form.

Expected Output / Result

Expected Output:
After clicking the button:
Mouse button clicked!

After pressing a key in the input:
Keyboard key pressed!

After entering Ravi and submitting the form:
Welcome Ravi

14. XML Student Details Validated Using DTD

Question:

Create an XML document to store student details such as student ID, name, branch, and marks. Write a DTD to validate the document.

Answer Code — File 1 (save as student.xml):

```
<?xml version="1.0"?>
<!DOCTYPE student SYSTEM "student.dtd">
<student>
  <id>101</id>
  <name>Ravi</name>
  <branch>CSE</branch>
  <marks>85</marks>
</student>
```

Answer Code — File 2 (save as student.dtd):

```
<!ELEMENT student (id, name, branch, marks)>
<!ELEMENT id (#PCDATA)>
<!ELEMENT name (#PCDATA)>
<!ELEMENT branch (#PCDATA)>
<!ELEMENT marks (#PCDATA)>
```

How to execute/validate:

1. Save student.xml and student.dtd in the same folder.
2. Open student.xml in an XML editor or IDE that supports DTD validation and validate it.
3. Ensure child elements appear in the specified order: id, name, branch, marks.

Note: A browser may display XML without performing DTD validation; use an XML validator or IDE validation feature.

Expected Output / Result

Expected Result:

If student.xml matches student.dtd, validation succeeds without errors.

Student data in the XML:

ID: 101

Name: Ravi

Branch: CSE

Marks: 85

15. XML Employee Details Validated Using XSD

Question:

Create an XML document containing employee details and validate it using an XML Schema Definition (XSD).

Answer Code — File 1 (save as employee.xml):

```
<?xml version="1.0"?>
<employee xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
     xsi:noNamespaceSchemaLocation="employee.xsd">
  <id>101</id>
  <name>Ravi</name>
  <department>CSE</department>
  <salary>30000</salary>
</employee>
```

Answer Code — File 2 (save as employee.xsd):

```
<?xml version="1.0"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
    <xs:element name="employee">
        <xs:complexType>
            <xs:sequence>
                <xs:element name="id" type="xs:int"/>
                <xs:element name="name" type="xs:string"/>
                <xs:element name="department" type="xs:string"/>
                <xs:element name="salary" type="xs:decimal"/>
            </xs:sequence>
        </xs:complexType>
    </xs:element>
</xs:schema>
```

How to execute/validate:

1. Save employee.xml and employee.xsd in the same folder.

2. Validate employee.xml against employee.xsd using an XML validator or IDE with XSD support.

3. Check that element names, order, and data types match the schema.

Expected Output / Result

Expected Result:

If employee.xml matches employee.xsd, validation succeeds without errors.

Employee data in the XML:

ID: 101

Name: Ravi

Department: CSE

Salary: 30000

16. Java Program to Parse XML and Display Student Details

Question:

Write a Java program to parse an XML document to display the details of all student elements.

Answer Code — File 1 (save as students.xml):

```
<?xml version="1.0"?>
<students>
    <student>
    <id>101</id>
    <name>Ravi</name>
    <branch>CSE</branch>
    </student>
    <student>
    <id>102</id>
    <name>Raju</name>
    <branch>ECE</branch>
    </student>
</students>
```

Answer Code — File 2 (save as StudentParser.java):

```
import javax.xml.parsers.*;
import org.w3c.dom.*;


public class StudentParser {
    public static void main(String[] args) throws Exception {
        DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
        DocumentBuilder builder = factory.newDocumentBuilder();
        Document doc = builder.parse("students.xml");


        NodeList list = doc.getElementsByTagName("student");


        for (int i = 0; i < list.getLength(); i++) {
            Element s = (Element) list.item(i);


            String id = s.getElementsByTagName("id").item(0).getTextContent();
            String name = s.getElementsByTagName("name").item(0).getTextContent();
            String branch = s.getElementsByTagName("branch").item(0).getTextContent();


            System.out.println("ID: " + id);
            System.out.println("Name: " + name);
            System.out.println("Branch: " + branch);
```

```
            System.out.println("--------------------");
        }
    }
}
```

How to execute:

1. Save students.xml and StudentParser.java in the same folder.

2. Open a terminal in that folder.

3. Compile: javac StudentParser.java

4. Run: java StudentParser

The program uses the DOM parser to load the XML, finds every student element, and prints its child values.

Expected Output / Result

Expected Output:
ID: 101
Name: Ravi
Branch: CSE

ID: 102
Name: Raju
Branch: ECE

JDBC LAB PROGRAMS

Questions 17–22

Oracle Database • Java • Windows run commands

Before you start

- Install a JDK and make sure java and javac work in the terminal.
- Download the Oracle JDBC driver JAR (for example, ojdbc11.jar) from Oracle's official JDBC downloads page and place it in the same folder as your .java files.
- The examples use jdbc:oracle:thin:@localhost:1521/XEPDB1 and the Oracle user system. Change the service name, username, and password to match your setup.
- For each program, compile and run using the commands shown. On Windows, the classpath separator is a semicolon (;). The dot (.) means the current folder.
- Question 17 creates the STUDENT table. Run it first. Questions 18–22 assume the table exists. If you run Question 17 repeatedly, the table may already exist and inserts may duplicate primary keys.

Oracle JDBC driver download: https://www.oracle.com/database/technologies/appdev/jdbc-downloads.html

17. Develop a JDBC application to connect to a database, create a student table, insert student records, and display them using ResultSet.

File name: StudentJDBC.java

```
import java.sql.*;

public class StudentJDBC {
    public static void main(String[] args) {
        String url = "jdbc:oracle:thin:@localhost:1521/XEPDB1";
        String user = "system";
        String password = "your_password";

        try {
            Class.forName("oracle.jdbc.OracleDriver");
            Connection con = DriverManager.getConnection(url, user, password);
            Statement st = con.createStatement();

            st.executeUpdate(
                "CREATE TABLE STUDENT (" +
                "ID NUMBER PRIMARY KEY, " +
                "NAME VARCHAR2(50), " +
                "MARKS NUMBER(5,2))"
            );
            System.out.println("STUDENT table created.");

            String sql = "INSERT INTO STUDENT (ID, NAME, MARKS) VALUES (?, ?, ?)";
            PreparedStatement ps = con.prepareStatement(sql);

            ps.setInt(1, 1);
            ps.setString(2, "Rahul");
            ps.setDouble(3, 85.5);
            ps.executeUpdate();

            ps.setInt(1, 2);
            ps.setString(2, "Priya");
            ps.setDouble(3, 91.0);
            ps.executeUpdate();

            ps.setInt(1, 3);
            ps.setString(2, "Arun");
            ps.setDouble(3, 78.0);
            ps.executeUpdate();
```

```
            System.out.println("Student records inserted.");

            ResultSet rs = st.executeQuery("SELECT * FROM STUDENT ORDER BY ID");
            System.out.println("\nID\tNAME\tMARKS");
            while (rs.next()) {
                System.out.println(
                    rs.getInt("ID") + "\t" +
                    rs.getString("NAME") + "\t" +
                    rs.getDouble("MARKS")
                );
            }

            rs.close();
            ps.close();
            st.close();
            con.close();
        } catch (SQLException | ClassNotFoundException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Running commands (Windows PowerShell / CMD)

```
javac -cp ".;ojdbc11.jar" StudentJDBC.java
java -cp ".;ojdbc11.jar" StudentJDBC
```

Note: Run this program first. CREATE TABLE will fail if STUDENT already exists, and inserts will fail if the same primary-key IDs already exist. If needed, remove the old table using DROP TABLE STUDENT before rerunning (only if you can safely delete its data).

Sample Output

```
STUDENT table created.
Student records inserted.

ID    NAME    MARKS
1     Rahul   85.5
2     Priya   91.0
3     Arun    78.0
```

18. Write a JDBC program to accept a student ID and display the corresponding student details using a PreparedStatement.

File name: SearchStudent.java

```
import java.sql.*;
import java.util.Scanner;

public class SearchStudent {
    public static void main(String[] args) {
        String url = "jdbc:oracle:thin:@localhost:1521/XEPDB1";
        String user = "system";
        String password = "your_password";

        try (Scanner sc = new Scanner(System.in)) {
            Connection con = DriverManager.getConnection(url, user, password);

            System.out.print("Enter student ID: ");
            int id = sc.nextInt();

            String sql = "SELECT * FROM STUDENT WHERE ID = ?";
            PreparedStatement ps = con.prepareStatement(sql);
```

```
            ps.setInt(1, id);

            ResultSet rs = ps.executeQuery();

            if (rs.next()) {
                System.out.println("Student found:");
                System.out.println("ID: " + rs.getInt("ID"));
                System.out.println("Name: " + rs.getString("NAME"));
                System.out.println("Marks: " + rs.getDouble("MARKS"));
            } else {
                System.out.println("Student not found.");
            }

            rs.close();
            ps.close();
            con.close();
        } catch (SQLException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Running commands (Windows PowerShell / CMD)

```
javac -cp ".;ojdbc11.jar" SearchStudent.java
java -cp ".;ojdbc11.jar" SearchStudent
```

Sample Output

```
Enter student ID: 2
Student found:
ID: 2
Name: Priya
Marks: 91.0
```

19. Develop a JDBC application to update and delete student records based on the student ID.

File name: UpdateDeleteStudent.java

```
import java.sql.*;
import java.util.Scanner;

public class UpdateDeleteStudent {
    public static void main(String[] args) {
        String url = "jdbc:oracle:thin:@localhost:1521/XEPDB1";
        String user = "system";
        String password = "your_password";

        try (Scanner sc = new Scanner(System.in)) {
            Connection con = DriverManager.getConnection(url, user, password);

            System.out.println("1. Update student");
            System.out.println("2. Delete student");
            System.out.print("Enter choice: ");
            int choice = sc.nextInt();

            if (choice == 1) {
                System.out.print("Enter student ID: ");
                int id = sc.nextInt();
                sc.nextLine(); // consume newline

                System.out.print("Enter new name: ");
```

```
                String name = sc.nextLine();

                System.out.print("Enter new marks: ");
                double marks = sc.nextDouble();

                String sql = "UPDATE STUDENT SET NAME = ?, MARKS = ? WHERE ID = ?";
                PreparedStatement ps = con.prepareStatement(sql);
                ps.setString(1, name);
                ps.setDouble(2, marks);
                ps.setInt(3, id);

                int rows = ps.executeUpdate();
                System.out.println(rows > 0 ? "Student updated." : "Student ID not found.");
                ps.close();

            } else if (choice == 2) {
                System.out.print("Enter student ID to delete: ");
                int id = sc.nextInt();

                String sql = "DELETE FROM STUDENT WHERE ID = ?";
                PreparedStatement ps = con.prepareStatement(sql);
                ps.setInt(1, id);

                int rows = ps.executeUpdate();
                System.out.println(rows > 0 ? "Student deleted." : "Student ID not found.");
                ps.close();

            } else {
                System.out.println("Invalid choice.");
            }

            con.close();
        } catch (SQLException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Running commands (Windows PowerShell / CMD)

```
javac -cp ".;ojdbc11.jar" UpdateDeleteStudent.java
java -cp ".;ojdbc11.jar" UpdateDeleteStudent
```

Note: Use choice 1 to update a row or choice 2 to delete a row. Deleting a student permanently removes that row from the table.

Sample Output

```
Sample Run 1 — Update
1. Update student
2. Delete student
Enter choice: 1
Enter student ID: 2
Enter new name: Priya Sharma
Enter new marks: 95
Student updated.

Sample Run 2 — Delete
1. Update student
2. Delete student
Enter choice: 2
Enter student ID to delete: 3
Student deleted.
```

20. Write a JDBC program to demonstrate DatabaseMetaData and ResultSetMetaData by displaying database, table, and column information.

File name: MetaDataDemo.java

```
import java.sql.*;

public class MetaDataDemo {
    public static void main(String[] args) {
        String url = "jdbc:oracle:thin:@localhost:1521/XEPDB1";
        String user = "system";
        String password = "your_password";

        try {
            Connection con = DriverManager.getConnection(url, user, password);
            DatabaseMetaData dbmd = con.getMetaData();

            System.out.println("=== Database Information ===");
            System.out.println("Database: " + dbmd.getDatabaseProductName());
            System.out.println("Database Version: " + dbmd.getDatabaseProductVersion());
            System.out.println("Driver: " + dbmd.getDriverName());
            System.out.println("Connected User: " + dbmd.getUserName());

            System.out.println("\n=== Tables ===");
            try (ResultSet tables = dbmd.getTables(
                    null, user.toUpperCase(), "%", new String[] {"TABLE"})) {
                while (tables.next()) {
                    System.out.println("Table: " + tables.getString("TABLE_NAME"));
                }
            }

            System.out.println("\n=== STUDENT Column Information ===");
            try (Statement st = con.createStatement();
                 ResultSet rs = st.executeQuery("SELECT * FROM STUDENT WHERE 1 = 0")) {

                ResultSetMetaData rsmd = rs.getMetaData();
                System.out.println("Number of columns: " + rsmd.getColumnCount());

                for (int i = 1; i <= rsmd.getColumnCount(); i++) {
                    System.out.println(
                        "Column " + i +
                        ": " + rsmd.getColumnName(i) +
                        ", Type: " + rsmd.getColumnTypeName(i) +
                        ", Display size: " + rsmd.getColumnDisplaySize(i)
                    );
                }
            }

            con.close();
        } catch (SQLException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Running commands (Windows PowerShell / CMD)

```
javac -cp ".;ojdbc11.jar" MetaDataDemo.java
java -cp ".;ojdbc11.jar" MetaDataDemo
```

Note: DatabaseMetaData describes the database and JDBC driver. ResultSetMetaData describes the columns returned by a query. This program assumes the STUDENT table exists in the SYSTEM schema.

Sample Output

```
=== Database Information ===
Database: Oracle
Database Version: (depends on installed Oracle version)
Driver: Oracle JDBC driver
Connected User: SYSTEM

=== Tables ===
Table: STUDENT

=== STUDENT Column Information ===
Number of columns: 3
Column 1: ID, Type: NUMBER, Display size: (driver-dependent)
Column 2: NAME, Type: VARCHAR2, Display size: (driver-dependent)
Column 3: MARKS, Type: NUMBER, Display size: (driver-dependent)
```

21. Develop a JDBC program to demonstrate transaction management using commit() and rollback().

File name: TransactionDemo.java

```
import java.sql.*;

public class TransactionDemo {
    public static void main(String[] args) {
        String url = "jdbc:oracle:thin:@localhost:1521/XEPDB1";
        String user = "system";
        String password = "your_password";

        Connection con = null;

        try {
            con = DriverManager.getConnection(url, user, password);
            con.setAutoCommit(false);

            String sql = "INSERT INTO STUDENT (ID, NAME, MARKS) VALUES (?, ?, ?)";
            try (PreparedStatement ps = con.prepareStatement(sql)) {
                ps.setInt(1, 101);
                ps.setString(2, "Kiran");
                ps.setDouble(3, 88.5);
                ps.executeUpdate();

                ps.setInt(1, 102);
                ps.setString(2, "Meena");
                ps.setDouble(3, 92.0);
                ps.executeUpdate();
            }

            con.commit();
            System.out.println("Transaction committed successfully.");

        } catch (SQLException e) {
            System.out.println("Transaction failed: " + e.getMessage());

            if (con != null) {
                try {
                    con.rollback();
                    System.out.println("Transaction rolled back.");
                } catch (SQLException rollbackError) {
                    System.out.println("Rollback failed: " + rollbackError.getMessage());
                }
            }
        } finally {
            if (con != null) {
                try {
```

```
                    con.setAutoCommit(true);
                    con.close();
                } catch (SQLException e) {
                    System.out.println("Closing error: " + e.getMessage());
                }
            }
        }
    }
}
```

Running commands (Windows PowerShell / CMD)

```
javac -cp ".;ojdbc11.jar" TransactionDemo.java
java -cp ".;ojdbc11.jar" TransactionDemo
```

Note: This program assumes STUDENT exists and IDs 101 and 102 are unused. If either insert fails, rollback() cancels the uncommitted changes in this transaction. Use fresh IDs if you rerun it.

Sample Output

```
Transaction committed successfully.
```

22. Write a JDBC program to demonstrate a scrollable ResultSet and a CachedRowSet by navigating through student records.

File name: ScrollableResultSetDemo.java

```
import java.sql.*;
import javax.sql.rowset.*;

public class ScrollableResultSetDemo {
    public static void main(String[] args) {
        String url = "jdbc:oracle:thin:@localhost:1521/XEPDB1";
        String user = "system";
        String password = "your_password";

        try {
            Connection con = DriverManager.getConnection(url, user, password);

            // 1. Scrollable ResultSet
            System.out.println("=== Scrollable ResultSet ===");

            Statement st = con.createStatement(
                ResultSet.TYPE_SCROLL_INSENSITIVE,
                ResultSet.CONCUR_READ_ONLY
            );

            ResultSet rs = st.executeQuery("SELECT * FROM STUDENT ORDER BY ID");

            if (rs.last())
                System.out.println("Last Student: " + rs.getString("NAME"));

            if (rs.first())
                System.out.println("First Student: " + rs.getString("NAME"));

            if (rs.absolute(2))
                System.out.println("Second Student: " + rs.getString("NAME"));

            if (rs.previous())
                System.out.println("Previous Student: " + rs.getString("NAME"));

            rs.close();
            st.close();
```

```
            // 2. CachedRowSet
            System.out.println("\n=== CachedRowSet ===");

            CachedRowSet crs =
                RowSetProvider.newFactory().createCachedRowSet();

            crs.setUrl(url);
            crs.setUsername(user);
            crs.setPassword(password);
            crs.setCommand("SELECT * FROM STUDENT ORDER BY ID");
            crs.execute(); // Load rows into memory

            con.close(); // CachedRowSet can now be used without this connection

            if (crs.last())
                System.out.println("Last Student: " + crs.getString("NAME"));

            if (crs.first())
                System.out.println("First Student: " + crs.getString("NAME"));

            if (crs.absolute(2))
                System.out.println("Second Student: " + crs.getString("NAME"));

            crs.close();

        } catch (SQLException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Running commands (Windows PowerShell / CMD)

```
javac -cp ".;ojdbc11.jar" ScrollableResultSetDemo.java
java -cp ".;ojdbc11.jar" ScrollableResultSetDemo
```

Note: Scrollable ResultSet supports first(), last(), previous(), and absolute(). CachedRowSet stores rows in memory, so you can navigate after closing the original Connection. The Java JDK includes the rowset API; ojdbc11.jar is still needed for Oracle connectivity.

Sample Output

```
=== Scrollable ResultSet ===
Last Student: Arun
First Student: Rahul
Second Student: Priya
Previous Student: Rahul

=== CachedRowSet ===
Last Student: Arun
First Student: Rahul
Second Student: Priya
```

Quick revision: important methods

Method / Concept	Meaning
executeQuery()	Runs a SELECT statement and returns a ResultSet.
executeUpdate()	Runs INSERT, UPDATE, DELETE, and many DDL statements; returns affected-row count for DML.
PreparedStatement	Uses ? placeholders and binds values with methods such as setInt() and setString().

`ResultSet.next()`	Moves to the next row and returns false when there are no more rows.
`DatabaseMetaData`	Provides information about the database and JDBC driver.
`ResultSetMetaData`	Provides information about columns in a ResultSet.
`commit()`	Makes the current transaction changes permanent.
`rollback()`	Cancels uncommitted changes in the current transaction.
`Scrollable ResultSet`	Supports navigation in different directions through query results.
`CachedRowSet`	Caches result rows in memory for disconnected use.

Important setup reminder: Replace your_password in each program. If your Oracle database uses a different service name, update the JDBC URL. Compile and run each file from the directory containing both the .java file and ojdbc11.jar.

Note: These examples use Jakarta Servlets (jakarta.servlet.*), suitable for Tomcat 10.1. Create each Java class in WEB-INF/classes, compile it with Tomcat's servlet-api.jar, and add the mappings shown in web.xml. HTML files should be placed in the web application's root folder.

23. Student Marks, Total, Average and Grade

student.html

```
<html>
<body>
    <h2>Student Marks</h2>
    <form action="StudentServlet" method="post">
        Name: <input type="text" name="name"><br><br>
        Marks 1: <input type="number" name="m1"><br><br>
        Marks 2: <input type="number" name="m2"><br><br>
        Marks 3: <input type="number" name="m3"><br><br>
        <input type="submit" value="Calculate">
    </form>
</body>
</html>
```

StudentServlet.java

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class StudentServlet extends HttpServlet {
    public void doPost(HttpServletRequest req,
                       HttpServletResponse res)
                       throws IOException {

        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        String name = req.getParameter("name");
        int m1 = Integer.parseInt(req.getParameter("m1"));
        int m2 = Integer.parseInt(req.getParameter("m2"));
        int m3 = Integer.parseInt(req.getParameter("m3"));

        int total = m1 + m2 + m3;
        double avg = total / 3.0;
        String grade;

        if (avg >= 90)
            grade = "A";
```

```
        else if (avg >= 75)
            grade = "B";
        else if (avg >= 50)
            grade = "C";
        else
            grade = "Fail";

        out.println("<h2>Student Result</h2>");
        out.println("Name: " + name + "<br>");
        out.println("Total: " + total + "<br>");
        out.println("Average: " + avg + "<br>");
        out.println("Grade: " + grade);
    }
}
```

24. Servlet Life Cycle

LifeCycleServlet.java

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class LifeCycleServlet extends HttpServlet {

    public void init() throws ServletException {
        System.out.println("Servlet initialized");
    }

    public void doGet(HttpServletRequest req,
                      HttpServletResponse res)
                      throws IOException {

        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        out.println("<h2>Servlet Life Cycle</h2>");
        out.println("<p>Service method executed using doGet()</p>");
    }

    public void destroy() {
        System.out.println("Servlet destroyed");
    }
}
```

Output in the browser:

```
Servlet Life Cycle
Service method executed using doGet()
```

Output in the server console:

```
Servlet initialized
Servlet destroyed
```

`init()` runs when the servlet is initialized, `doGet()` handles GET requests through the servlet's `service()` method, and `destroy()` runs when the servlet is taken out of service.

25. Login Application Using Servlet

login.html

```
<html>
<body>
    <h2>Login Page</h2>
    <form action="LoginServlet" method="post">
        Username: <input type="text" name="username"><br><br>
        Password: <input type="password" name="password"><br><br>
        <input type="submit" value="Login">
    </form>
</body>
</html>
```

LoginServlet.java

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class LoginServlet extends HttpServlet {
    public void doPost(HttpServletRequest req,
                       HttpServletResponse res)
                       throws IOException {

        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        String user = req.getParameter("username");
        String pass = req.getParameter("password");

        if (user.equals("admin") && pass.equals("123")) {
            out.println("<h2>Welcome " + user + "</h2>");
        } else {
            out.println("<h2>Invalid Username or Password</h2>");
        }
    }
}
```

Output for valid login (username: `admin`, password: `123`):

```
Welcome admin
```

Output for invalid login:

```
Invalid Username or Password
```

web.xml for all three programs

Place this file inside the WEB-INF folder. You can use the same web.xml for all three programs if they are in the same web application.

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
         version="6.0">

    <servlet>
        <servlet-name>StudentServlet</servlet-name>
        <servlet-class>StudentServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>StudentServlet</servlet-name>
        <url-pattern>/StudentServlet</url-pattern>
    </servlet-mapping>

    <servlet>
        <servlet-name>LifeCycleServlet</servlet-name>
        <servlet-class>LifeCycleServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>LifeCycleServlet</servlet-name>
        <url-pattern>/LifeCycleServlet</url-pattern>
    </servlet-mapping>

    <servlet>
        <servlet-name>LoginServlet</servlet-name>
        <servlet-class>LoginServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>LoginServlet</servlet-name>
        <url-pattern>/LoginServlet</url-pattern>
    </servlet-mapping>

</web-app>
```

Note: These are simple practical-exam versions. The student marks program assumes three marks out of 100 each and uses a basic grading scheme.

26. Demonstrate ServletConfig and ServletContext

Here is a simple Servlet application demonstrating ServletConfig and ServletContext for reading initialization parameters.

1. ConfigContextServlet.java

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class ConfigContextServlet extends HttpServlet {
    public void doGet(HttpServletRequest req,
                      HttpServletResponse res)
                      throws IOException {

        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        ServletConfig config = getServletConfig();
        String name = config.getInitParameter("username");

        ServletContext context = getServletContext();
        String college = context.getInitParameter("college");

        out.println("<h2>ServletConfig: " + name + "</h2>");
        out.println("<h2>ServletContext: " + college + "</h2>");
    }
}
```

2. web.xml

Place this file inside the WEB-INF folder.

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
         version="6.0">

    <context-param>
        <param-name>college</param-name>
        <param-value>ABC College</param-value>
    </context-param>

    <servlet>
        <servlet-name>ConfigContextServlet</servlet-name>
        <servlet-class>ConfigContextServlet</servlet-class>

        <init-param>
            <param-name>username</param-name>
            <param-value>Rahul</param-value>
```

```
        </init-param>
    </servlet>

    <servlet-mapping>
        <servlet-name>ConfigContextServlet</servlet-name>
        <url-pattern>/ConfigContextServlet</url-pattern>
    </servlet-mapping>

</web-app>
```

Output

```
ServletConfig: Rahul
ServletContext: ABC College
```

Explanation

- ServletConfig: Reads initialization parameters specific to a particular servlet using `getInitParameter()`.
- ServletContext: Reads initialization parameters shared across the entire web application using `getInitParameter()`.
- `<init-param>` defines a servlet-specific parameter.
- `<context-param>` defines an application-wide parameter.

27. Store a Student's Name in an HTTP Session

Here is a simple Servlet application to store a student's name in an HTTP session and display it on another page.

1. StudentServlet.java

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class StudentServlet extends HttpServlet {
    public void doGet(HttpServletRequest req,
                      HttpServletResponse res)
                      throws IOException {

        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        HttpSession session = req.getSession();
        session.setAttribute("name", "Rahul");

        out.println("<h2>Student name stored</h2>");
        out.println("<a href='DisplayServlet'>Display Name</a>");
```

```
    }
}
```

2. DisplayServlet.java

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class DisplayServlet extends HttpServlet {
    public void doGet(HttpServletRequest req,
                      HttpServletResponse res)
                      throws IOException {

        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        HttpSession session = req.getSession(false);

        if (session != null) {
            out.println("<h2>Student Name: " +
                        session.getAttribute("name") + "</h2>");
        } else {
            out.println("<h2>Session not found</h2>");
        }
    }
}
```

3. web.xml

Place this file inside the WEB-INF folder.

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
         version="6.0">

    <servlet>
        <servlet-name>StudentServlet</servlet-name>
        <servlet-class>StudentServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>StudentServlet</servlet-name>
        <url-pattern>/StudentServlet</url-pattern>
    </servlet-mapping>

    <servlet>
        <servlet-name>DisplayServlet</servlet-name>
        <servlet-class>DisplayServlet</servlet-class>
    </servlet>
    <servlet-mapping>
```

```
        <servlet-name>DisplayServlet</servlet-name>
        <url-pattern>/DisplayServlet</url-pattern>
    </servlet-mapping>

</web-app>
```

Output

First page:

```
Student name stored
Display Name
```

Click Display Name.

Second page:

```
Student Name: Rahul
```

28. Demonstrate Session Tracking Using Cookies

Here is a simple Servlet application to demonstrate session tracking using cookies.

1. CookieServlet.java

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class CookieServlet extends HttpServlet {
    public void doGet(HttpServletRequest req,
                      HttpServletResponse res)
                      throws IOException {

        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        Cookie c = new Cookie("username", "Rahul");
        res.addCookie(c);

        out.println("<h2>Cookie Created Successfully</h2>");
        out.println("<a href='WelcomeServlet'>Continue</a>");
    }
}
```

2. WelcomeServlet.java

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class WelcomeServlet extends HttpServlet {
    public void doGet(HttpServletRequest req,
                      HttpServletResponse res)
                      throws IOException {

        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        Cookie[] cookies = req.getCookies();

        if (cookies != null) {
            for (Cookie c : cookies) {
                if (c.getName().equals("username")) {
                    out.println("<h2>Welcome " +
                                c.getValue() + "</h2>");
                }
            }
        } else {
            out.println("<h2>Cookie not found</h2>");
        }
    }
}
```

3. web.xml

Place this file inside the WEB-INF folder.

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
         version="6.0">

    <servlet>
        <servlet-name>CookieServlet</servlet-name>
        <servlet-class>CookieServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>CookieServlet</servlet-name>
        <url-pattern>/CookieServlet</url-pattern>
    </servlet-mapping>

    <servlet>
        <servlet-name>WelcomeServlet</servlet-name>
        <servlet-class>WelcomeServlet</servlet-class>
```

```
    </servlet>
    <servlet-mapping>
        <servlet-name>WelcomeServlet</servlet-name>
        <url-pattern>/WelcomeServlet</url-pattern>
    </servlet-mapping>

</web-app>
```

Output

First page:

```
Cookie Created Successfully
Continue
```

Click Continue.

Second page:

```
Welcome Rahul
```

Common Setup Notes

1. Put HTML files in: check/ (the web application root).

2. Put Java source files in: check/WEB-INF/classes/ (for this simple no-package setup).

3. Compile each Java file using the Tomcat 10.1 servlet API. Example PowerShell commands:

```
$base = "C:\Program Files\Apache Software Foundation\Tomcat 10.1\webapps\check"
$jar = "C:\Program Files\Apache Software Foundation\Tomcat 10.1\lib\servlet-api.jar"
javac -cp $jar -d "$base\WEB-INF\classes" "$base\WEB-INF\classes\MarksServlet.java"
```

Repeat the javac command for each servlet class. For the session program compile both StoreNameServlet.java and DisplayNameServlet.java; for the cookie program compile both SetCookieServlet.java and ReadCookieServlet.java.

4. Add the relevant servlet and servlet-mapping entries to WEB-INF/web.xml. If you combine all programs in one web.xml, use unique servlet-name values and URL patterns.

5. Restart or reload the application if needed, then visit the URL listed under each program. The examples assume the application context is /check.

Question 29

Write a Servlet application to demonstrate URL rewriting using response.encodeURL() and session tracking.

Servlet Program: URL Rewriting Using response.encodeURL()

Aim: To demonstrate URL rewriting for session tracking in a Servlet application.

1. LoginServlet.java

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class LoginServlet extends HttpServlet {
    public void doGet(HttpServletRequest req,
                      HttpServletResponse res)
                      throws IOException {

        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        HttpSession session = req.getSession();
        session.setAttribute("username", "Rahul");

        String url = res.encodeURL("WelcomeServlet");

        out.println("<h2>Login Successful</h2>");
        out.println("<a href='" + url + "'>Continue</a>");
    }
}
```

2. WelcomeServlet.java

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class WelcomeServlet extends HttpServlet {
    public void doGet(HttpServletRequest req,
                      HttpServletResponse res)
                      throws IOException {

        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        HttpSession session = req.getSession(false);

        if (session != null) {
            String name = (String) session.getAttribute("username");
            out.println("<h2>Welcome " + name + "</h2>");
            out.println("Session tracking using URL rewriting.");
```

```
        } else {
            out.println("Session expired. Please login again.");
        }
    }
}
```

3. web.xml

Place this file inside the WEB-INF folder.

```
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
         version="6.0">

    <servlet>
        <servlet-name>LoginServlet</servlet-name>
        <servlet-class>LoginServlet</servlet-class>
    </servlet>

    <servlet-mapping>
        <servlet-name>LoginServlet</servlet-name>
        <url-pattern>/LoginServlet</url-pattern>
    </servlet-mapping>

    <servlet>
        <servlet-name>WelcomeServlet</servlet-name>
        <servlet-class>WelcomeServlet</servlet-class>
    </servlet>

    <servlet-mapping>
        <servlet-name>WelcomeServlet</servlet-name>
        <url-pattern>/WelcomeServlet</url-pattern>
    </servlet-mapping>

</web-app>
```

Sample output

First page:

Login Successful

Continue

After clicking Continue:

Welcome Rahul

Session tracking using URL rewriting.

Question 30

Develop a Servlet application to demonstrate hidden form fields for passing student information from one page to another.

Servlet Program: Hidden Form Fields

Aim: To develop a Servlet application to pass student information from one page to another using hidden form fields.

1. student.html

```
<!DOCTYPE html>
<html>
<body>
    <h2>Student Details</h2>

    <form action="StudentServlet" method="post">
        Name: <input type="text" name="name"><br><br>
        Roll No: <input type="text" name="rollno"><br><br>
        <input type="submit" value="Submit">
    </form>
</body>
</html>
```

2. StudentServlet.java

This servlet receives the student details and passes them to the next page using hidden fields.

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class StudentServlet extends HttpServlet {
    public void doPost(HttpServletRequest req,
                       HttpServletResponse res)
                       throws IOException {

        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        String name = req.getParameter("name");
        String rollno = req.getParameter("rollno");

        out.println("<h2>Student Details Submitted</h2>");
        out.println("<form action='DisplayServlet' method='post'>");

        out.println("<input type='hidden' name='name' value='" + name + "'>");
        out.println("<input type='hidden' name='rollno' value='" + rollno + "'>");

        out.println("<input type='submit' value='View Details'>");
        out.println("</form>");
    }
}
```

3. DisplayServlet.java

This servlet receives and displays the information passed through hidden fields.

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class DisplayServlet extends HttpServlet {
    public void doPost(HttpServletRequest req,
                       HttpServletResponse res)
                       throws IOException {

        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        String name = req.getParameter("name");
        String rollno = req.getParameter("rollno");

        out.println("<h2>Student Information</h2>");
        out.println("Name: " + name + "<br>");
        out.println("Roll No: " + rollno);
    }
}
```

4. web.xml

Place this file inside WEB-INF.

```
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
         version="6.0">

    <servlet>
        <servlet-name>StudentServlet</servlet-name>
        <servlet-class>StudentServlet</servlet-class>
    </servlet>

    <servlet-mapping>
        <servlet-name>StudentServlet</servlet-name>
        <url-pattern>/StudentServlet</url-pattern>
    </servlet-mapping>

    <servlet>
        <servlet-name>DisplayServlet</servlet-name>
        <servlet-class>DisplayServlet</servlet-class>
    </servlet>

    <servlet-mapping>
        <servlet-name>DisplayServlet</servlet-name>
        <url-pattern>/DisplayServlet</url-pattern>
    </servlet-mapping>

</web-app>
```

Sample output

```
Page 1 – Student Details

Student Details

Name: Rahul
Roll No: 101

[Submit]

Page 2 – Student Details Submitted

Student Details Submitted

[View Details]

Page 3 – Student Information

Student Information

Name: Rahul
Roll No: 101
```

Command to Execute

```
$base = "C:\Program Files\Apache Software Foundation\Tomcat 10.1\webapps\check"
$jar = "C:\Program Files\Apache Software Foundation\Tomcat 10.1\lib\servlet-api.jar"
javac -cp $jar -d "$base\WEB-INF\classes" "$base\WEB-INF\classes\HiddenFieldServlet.java"
javac -cp $jar -d "$base\WEB-INF\classes" "$base\WEB-INF\classes\ShowHiddenServlet.java"
```

Run: http://localhost:8080/check/student-hidden.html

Question 31

Create a login and logout application using HttpSession. After logout, invalidate the session.

1. LoginServlet.java

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class LoginServlet extends HttpServlet {
    public void doPost(HttpServletRequest req,
                       HttpServletResponse res)
                       throws IOException {

        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        String user = req.getParameter("username");
        String pass = req.getParameter("password");

        if (user.equals("admin") && pass.equals("123")) {
            HttpSession session = req.getSession();
            session.setAttribute("username", user);

            out.println("<h2>Login Successful</h2>");
            out.println("<a href='WelcomeServlet'>Continue</a>");
        } else {
            out.println("<h2>Invalid Login</h2>");
        }
    }
}
```

2. WelcomeServlet.java

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class WelcomeServlet extends HttpServlet {
    public void doGet(HttpServletRequest req,
                      HttpServletResponse res)
                      throws IOException {

        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        HttpSession session = req.getSession(false);
```

```
        if (session != null) {
            out.println("<h2>Welcome " +
                session.getAttribute("username") + "</h2>");
            out.println("<a href='LogoutServlet'>Logout</a>");
        } else {
            out.println("<h2>Please login first</h2>");
        }
    }
}
```

3. LogoutServlet.java

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class LogoutServlet extends HttpServlet {
    public void doGet(HttpServletRequest req,
                      HttpServletResponse res)
                      throws IOException {

        HttpSession session = req.getSession(false);

        if (session != null) {
            session.invalidate();
        }

        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        out.println("<h2>Logout Successful</h2>");
    }
}
```

4. login.html

```
<html>
<body>
    <h2>Login Page</h2>
    <form action="LoginServlet" method="post">
        Username: <input type="text" name="username"><br><br>
        Password: <input type="password" name="password"><br><br>
        <input type="submit" value="Login">
    </form>
</body>
</html>
```

5. web.xml

Place this file inside `WEB-INF`.

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
         version="6.0">

    <servlet>
        <servlet-name>Login</servlet-name>
        <servlet-class>LoginServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>Login</servlet-name>
        <url-pattern>/LoginServlet</url-pattern>
    </servlet-mapping>

    <servlet>
        <servlet-name>Welcome</servlet-name>
        <servlet-class>WelcomeServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>Welcome</servlet-name>
        <url-pattern>/WelcomeServlet</url-pattern>
    </servlet-mapping>

    <servlet>
        <servlet-name>Logout</servlet-name>
        <servlet-class>LogoutServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>Logout</servlet-name>
        <url-pattern>/LogoutServlet</url-pattern>
    </servlet-mapping>

</web-app>
```

Output

1. Enter username `admin` and password `123`.
2. Login Successful — click Continue.
3. Welcome admin — click Logout.
4. Logout Successful.

Important methods:

- `getSession()` — creates or retrieves a session.
- `setAttribute()` — stores the username.
- `getAttribute()` — retrieves the username.

- `invalidate()` — destroys the session during logout.

Command Reference

For each question, the document includes the question, answer code, web.xml mappings, and compile command. Put HTML files in the check folder and Java source files in check\WEB-INF\classes. Combine all mappings in one WEB-INF\web.xml and ensure each servlet-name and URL pattern is unique.

Common PowerShell setup:

```
$base = "C:\Program Files\Apache Software Foundation\Tomcat 10.1\webapps\check"
$jar = "C:\Program Files\Apache Software Foundation\Tomcat 10.1\lib\servlet-api.jar"
```

General compile pattern:

```
javac -cp $jar -d "$base\WEB-INF\classes" "$base\WEB-INF\classes\ExampleServlet.java"
```